



SecPass

Documentação Técnica Completa — Arquitetura, Segurança e Funcionamento

Gerenciador de senhas multiplataforma (Android, iOS e macOS)

Cortexis Tech

Documento gerado em 9 de setembro de 2026

1. Visão Geral

SecPass é um gerenciador de senhas multiplataforma com dois clientes que compartilham a mesma lógica de cofre, criptografia e sincronização: um aplicativo **mobile** (Android e iOS, construído em React Native/Expo) e um aplicativo **desktop** para macOS (construído em Electron). Os dois entendem o mesmo formato de cofre cifrado, o que permite que um usuário crie um item em um Mac e o veja, já decifrado, no celular — e vice-versa — através de um backend de sincronização à sua escolha.

O produto foi desenhado em torno de um princípio central: **criptografia de ponta a ponta (E2E) client-side**. O servidor de sincronização (Google Drive ou iCloud) nunca vê uma senha em texto claro — ele armazena apenas blobs cifrados. A chave de criptografia é derivada localmente, em cada dispositivo, a partir do e-mail e da senha de acesso do usuário, e nunca trafega pela rede.

1.1 Plataformas e propósito de cada uma

Cliente	Tecnologia	Uso típico
Android	React Native + Expo (SDK 57)	App principal, biometria via <code>expo-local-authentication</code>
iOS	React Native + Expo + módulo nativo Swift	Mesmo código React, mas com acesso nativo ao CloudKit via módulo Swift próprio
macOS	Electron (Chromium + Node.js)	App desktop independente, reaproveitando os módulos JS de criptografia/sync do mobile

1.2 Modelo de sincronização escolhido pelo usuário

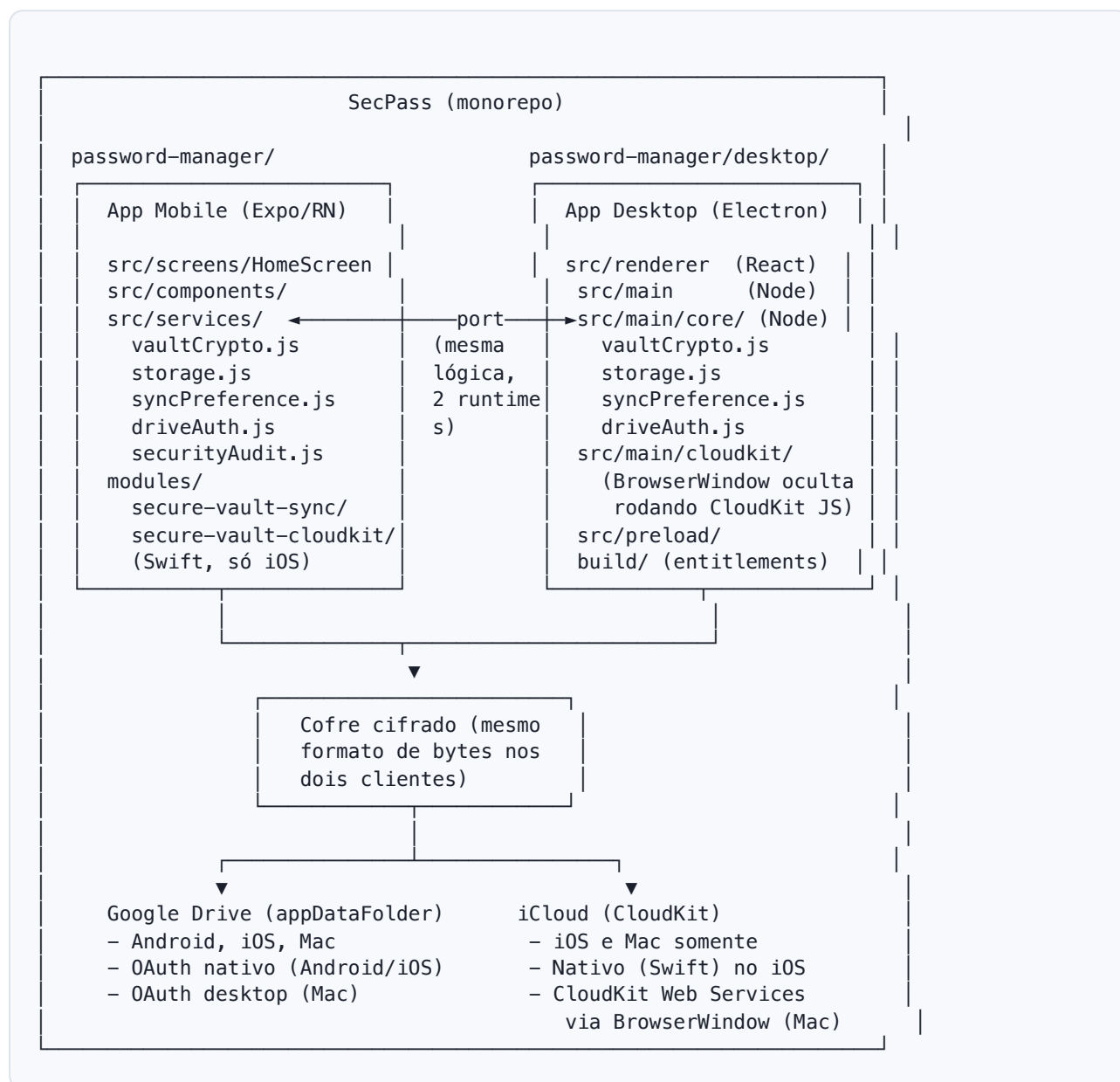
Uma decisão de produto importante tomada durante o desenvolvimento: em vez de escolher automaticamente um backend de nuvem, o app deixa o **usuário decidir, por aparelho**, qual ecossistema de sincronização usar:

- **Google Drive** (pasta oculta `appDataFolder`, invisível ao usuário no Drive normal) — funciona entre Android, iOS e Mac. É a opção recomendada para quem usa uma mistura de sistemas (ecossistema "híbrido").
- **iCloud (CloudKit)** — funciona apenas entre iPhone e Mac. É a opção recomendada para quem usa exclusivamente produtos Apple, evitando depender de uma conta Google.

Essa escolha é armazenada localmente por dispositivo (`syncPreference.js`) e pode ser trocada a qualquer momento pela tela de "Sincronização", com migração dos dados existentes para o novo backend.

2. Arquitetura Geral

O projeto é um monorepo (sem uso de submódulos/workspaces formais) onde a pasta raiz é o app mobile, e a pasta `desktop/` contém o app Electron. O desktop importa e reaproveita diretamente vários arquivos de lógica de negócio do mobile (adaptados para rodar em Node.js puro, sem APIs do React Native).



2.1 Por que o desktop usa CloudKit "Web Services" e não o framework nativo

A Apple não expõe o framework nativo `CloudKit` para aplicações que não sejam apps Swift/Objective-C empacotados como app Mac "de verdade" via Xcode com entitlements de iCloud — um app Electron não tem esse acesso. A solução adotada foi usar a biblioteca

`cloudkit.js` da Apple (a mesma usada em páginas web que integram com iCloud), rodando dentro de uma `BrowserWindow` do Electron oculta (não visível ao usuário), controlada via uma ponte de IPC (processo principal $\rightleftharpoons$ janela oculta $\rightleftharpoons$ processo principal). Essa abordagem funciona, mas não é um cenário oficialmente documentado pela Apple para esse fim — está sinalizado como algo a validar com mais uso real (ver seção 12).

3. Estrutura de Pastas

3.1 App mobile (raiz do repositório)

Pasta/Arquivo	Conteúdo
<code>src/app/</code>	Rotas do Expo Router (entrada do app)
<code>src/screens/HomeScreen.js</code>	Tela principal — login, cofre, listagem de itens, todos os modais (sincronização, histórico de segurança, etc.). Arquivo central do app, ~2.500 linhas.
<code>src/components/</code>	Componentes reutilizáveis: cartão de senha (<code>PasswordCard.js</code>), formulário (<code>PasswordForm.tsx</code>), busca, logo, créditos
<code>src/services/</code>	Toda a lógica de negócio: criptografia, storage, autenticação, auditoria (detalhado na seção 4 em diante)
<code>src/hooks/</code> , <code>src/utils/</code>	Hooks e utilitários de apoio
<code>modules/secure-vault-sync/</code>	Módulo nativo Expo (Swift) para autenticação/token do Google Drive no iOS
<code>modules/secure-vault-cloudkit/</code>	Módulo nativo Expo (Swift) que fala diretamente com o framework CloudKit nativo do iOS
<code>plugins/</code>	Config plugins customizados do Expo (ver seção 11)
<code>patches/</code>	Patches permanentes via <code>patch-package</code> (ver seção 11)
<code>__tests__/</code>	Suíte de testes Jest (210 testes, 17 arquivos)
<code>android/</code> , <code>ios/</code>	Projetos nativos gerados pelo <code>expo prebuild</code> (não editados manualmente — regeneráveis)

3.2 App desktop (`desktop/`)

Pasta/Arquivo	Conteúdo
<code>src/main/index.js</code>	Processo principal do Electron — cria janelas, registra handlers de IPC
<code>src/main/core/</code>	Portas Node.js dos mesmos serviços do mobile (criptografia, storage, sync, etc.)
<code>src/main/cloudkit/</code>	Janela oculta + ponte IPC para rodar CloudKit JS (ver seção 2.1)
<code>src/main/secureStore.js</code>	Armazenamento seguro local usando o Keychain do macOS (via <code>safeStorage</code> do Electron)

<code>src/preload/</code>	Scripts de preload — ponte segura e controlada entre o processo principal e o renderer
<code>src/renderer/</code>	Interface React do app desktop
<code>build/entitlements.mac.plist</code>	Entitlements para assinatura com hardened runtime
<code>__tests__/</code>	Suíte de testes Jest própria (50 testes, 4 arquivos)

4. O Cofre e o Modelo de Criptografia

4.1 Derivação de chave

Quando o usuário cria ou desbloqueia uma conta local, a senha de acesso e o e-mail são usados para derivar as chaves de criptografia via **PBKDF2-SHA256 com 600.000 iterações** — um custo computacional deliberadamente alto para dificultar ataques de força bruta offline caso um atacante obtenha uma cópia do cofre cifrado. Dessa derivação nascem várias sub-chaves com propósitos distintos (chave de cifragem, chave de HMAC/verificação, chave de selo do log de auditoria), cada uma derivada de forma independente (via HMAC) para que o comprometimento de uma não comprometa as outras.

4.2 Cifragem dos itens (envelope)

Cada item do cofre (uma credencial: site, usuário, senha, notas) é cifrado individualmente com **AES-256-GCM** — uma cifra autenticada (AEAD), que garante não só sigilo como também integridade: qualquer alteração de um único byte do texto cifrado é detectada na decifragem. O formato do envelope evoluiu em três versões ao longo do projeto:

Versão	Cifra	Observação
v1 (legado)	AES-256-CBC + HMAC-SHA256 separado	Formato antigo, mantido só para conseguir ler cofres muito antigos
v2	AES-256-GCM	Passou a usar uma cifra autenticada única (AEAD) em vez de CBC+HMAC
v3 (atual)	AES-256-GCM com AAD vinculada	Além da cifragem, vincula <code>id</code> , data de revisão (<code>updatedAt</code>) e a flag de exclusão (<code>tombstone</code>) como <i>dados associados autenticados</i> (AAD) do AES-GCM

Por que a v3 existe: nas versões anteriores, um item cifrado válido podia, em tese, ser "reaproveitado" por alguém com acesso de escrita ao backend remoto — substituindo um item atual por uma cópia antiga (porém genuína, cifrada com a mesma senha), sem que a decifragem detectasse nada de errado, já que o conteúdo cifrado em si continuava íntegro. Ao vincular a data de revisão e o estado de exclusão na AAD, qualquer tentativa de "voltar no tempo" um item (rollback) ou trocar sua identidade faz a verificação do AEAD falhar, e o item é rejeitado na decifragem. Essa correção foi implementada durante a auditoria de segurança adversarial descrita na seção 9.

4.3 Formato do cofre completo

O cofre é serializado como uma coleção de registros, cada um com:

```
{
  id: string,           // identificador único do item
  envelope: {...},      // dados cifrados (ver 4.2)
  updatedAt: number,    // timestamp da última revisão (texto claro)
  tombstone: boolean    // true = item excluído (soft-delete, para sincronizar exclus
```

Note que `updatedAt` e `tombstone` ficam em texto claro no registro (não dentro do envelope cifrado) — isso é necessário para que o algoritmo de merge (seção 4.4) e a sincronização consigam decidir qual versão de um item é mais recente *sem precisar decifrar nada*. Isso não vaza informação sensível (não revela o conteúdo da senha), só metadados de quando o item mudou pela última vez.

4.4 Sincronização e merge multi-dispositivo

Como não existe um servidor central que arbitra conflitos (o "servidor" é só um espaço de armazenamento cego — Google Drive ou CloudKit), o merge de itens entre dispositivos segue uma estratégia **last-write-wins por item**, comparando o campo `updatedAt` de cada lado (local vs. remoto) e mantendo o mais recente, implementada em `vaultMerge.js`. Itens excluídos em um dispositivo se propagam como "tombstones" (marcadores de exclusão) para os demais, em vez de serem removidos silenciosamente — isso evita que um dispositivo offline "ressuscite" um item que foi apagado em outro lugar.

5. Autenticação e Controle de Acesso

5.1 Conta local e senha de acesso

O SecPass não tem um "servidor de contas" — a "conta" é inteiramente local a cada dispositivo (email + hash/verificador da senha, guardado no Keychain/SecureStore do sistema operacional). A mesma combinação de email+senha em dois dispositivos diferentes gera a mesma chave de derivação, o que é o que permite a sincronização funcionar sem nunca enviar a senha para lugar nenhum.

5.2 Biometria

No mobile, o desbloqueio do cofre pode usar biometria (Face ID/Touch ID/impressão digital) via `expo-local-authentication`, como atalho para não digitar a senha de acesso toda vez — a senha continua sendo a raiz de tudo (a biometria desbloqueia o acesso ao dispositivo, não substitui a derivação da chave). No desktop, o mesmo padrão é usado com o Touch ID do Mac quando disponível.

5.3 Proteção contra força bruta local

Tentativas de senha incorretas incrementam um contador persistido em disco (`loginGuard.js`), que aciona bloqueios progressivos (throttle) com tempo de espera crescente após múltiplas falhas. Esse estado é salvo **antes** de atualizar a interface, para impedir que um atacante contorne o bloqueio simplesmente matando e reabrindo o app no meio de uma tentativa (uma das correções feitas na auditoria de segurança, seção 9).

5.4 Proteção contra captura de tela

No mobile, a tela é protegida contra captura (screenshot) e contra exibição de conteúdo sensível no "app switcher" do sistema (via `expo-screen-capture`) — essa proteção fica ativa desde a tela de login, não só depois do cofre estar desbloqueado, e só é desativada quando o componente é desmontado (não durante um simples logout dentro do app).

5.5 Limpeza automática da área de transferência

Quando o usuário copia uma senha para a área de transferência (clipboard), o valor é apagado automaticamente após 30 segundos, tanto no mobile quanto no desktop — reduzindo a janela de exposição caso o dispositivo seja comprometido ou emprestado logo depois de uma cópia.

6. Sincronização Entre Dispositivos

6.1 Google Drive

Usa a `appDataFolder` da Google Drive API — uma pasta especial, invisível na interface normal do Drive, acessível apenas pelo próprio app. No Android e iOS, a autenticação usa o SDK nativo `@react-native-google-signin/google-signin` (fluxo OAuth do próprio sistema). No desktop, como não existe esse SDK nativo para Electron, é usado um fluxo OAuth "para aplicativo de computador" próprio do Google Cloud.

6.2 iCloud (CloudKit)

No iOS, a integração é nativa, via o módulo Swift `secure-vault-cloudkit`, que fala diretamente com o framework `CloudKit` do sistema operacional. No desktop, como explicado na seção 2.1, a mesma função é obtida via CloudKit Web Services rodando numa janela oculta do Electron.

6.3 Escolha manual e migração de backend

A preferência de backend (`syncPreference.js`) é armazenada por dispositivo, não por conta — ou seja, o mesmo usuário pode escolher Google Drive no Android e iCloud no Mac, por exemplo, mas nesse caso os dois "mundos" não se sincronizam entre si (cada backend é isolado do outro). A troca de backend, feita pela tela de "Sincronização", roda uma migração que envia os itens do cofre atualmente decifrados na memória para o novo backend antes de trocar a preferência — não existe fallback automático silencioso: se o backend escolhido está indisponível, o app avisa o usuário em vez de tentar outro silenciosamente (decisão de design tomada para dar previsibilidade ao usuário sobre onde seus dados estão).

6.4 Confirmação ao encontrar um cofre remoto já existente (dispositivo novo)

Um caso específico recebeu atenção extra: quando alguém instala o app pela primeira vez num dispositivo novo (sem nenhum cofre local ainda) e informa um email/senha que já tem um cofre remoto associado, o app agora **mostra um resumo desse cofre** — quantidade de itens e data da última modificação — e pede confirmação explícita antes de baixá-lo e decifrá-lo.

Por que isso importa: num esquema de sincronização E2E sem um servidor que arbitra qual é a "versão verdadeira" dos dados (o mesmo desafio de fundo que aplicativos como o Signal resolvem com infraestrutura de servidor própria), um dispositivo completamente novo não tem uma cópia local para comparar — ele confia inteiramente no que está no backend remoto. Se alguém alguma vez teve acesso de escrita ao backend (por exemplo, um token OAuth vazado) e substituiu o conteúdo por uma cópia antiga, mas genuína, cifrada com a mesma senha, o dispositivo novo não teria como perceber isso sozinho. Mostrar o resumo (quantidade de itens, última modificação) não prova nada

criptograficamente, mas dá ao usuário uma chance de notar algo estranho (por exemplo, "só tem 2 itens, mas eu tinha 40" ou "a última modificação é de um ano atrás").

7. Auditoria e Segurança Adicional

7.1 Log de auditoria de segurança com selo à prova de adulteração

O app mantém um log local de eventos de segurança (login bem-sucedido, login falho, troca de backend de sincronização, exclusão do cofre, etc.), visível ao usuário na tela de "Histórico de segurança". Para detectar se esse log foi adulterado (por exemplo, um atacante com acesso físico ao dispositivo apagando a entrada que registra o próprio acesso indevido), o log é protegido por uma **cadeia de hashes HMAC**:

- Cada novo lote de eventos gera um hash encadeado (SHA-256) sobre os eventos, do mais antigo para o mais novo;
- Esse hash é assinado com uma chave HMAC derivada da própria senha do cofre (`deriveSealKey`), então só quem sabe a senha consegue gerar um selo válido;
- Um "id de fronteira" (`boundaryId` , o id do evento mais antigo no conjunto selado) permite distinguir uma remoção maliciosa de entradas de uma simples rotação natural do log (o log tem um limite máximo de eventos guardados, e os mais antigos são descartados de forma legítima com o tempo).

Na hora de desbloquear o cofre, o app verifica esse selo e reporta um dos três estados: `ok` (log íntegro), `tampered` (log foi alterado fora do fluxo normal do app) ou `inconclusive` (não há selo anterior para comparar, por exemplo na primeira vez).

7.2 Monitoramento de erros com redação de dados sensíveis

O app usa o Sentry para monitoramento de erros em produção. Antes de qualquer evento ser enviado, um filtro de redação (`errorMonitoring.ts`) remove ou mascara padrões que possam conter dados sensíveis — e-mails, nomes de usuário, endereços MAC, entre outros — para que um erro relatado nunca vazze acidentalmente informação pessoal do usuário para a ferramenta de terceiros.

7.3 Segurança específica do desktop

Mecanismo	Descrição
Armazenamento seguro local	<code>secureStore.js</code> usa o <code>safeStorage</code> do Electron (que por sua vez usa o Keychain do macOS). Se a criptografia do sistema operacional não estiver disponível, o app agora recusa salvar em texto claro em vez de degradar silenciosamente a segurança.
Isolamento de processos (sandbox)	As janelas do Electron rodam com o sandbox padrão ativado (não desabilitado), reduzindo a superfície de ataque caso o conteúdo renderizado seja comprometido.

Janela oculta do CloudKit	Restringe navegação apenas aos domínios da Apple necessários (<code>cdn.apple-cloudkit.com</code> , <code>idmsa.apple.com</code> , <code>appleid.apple.com</code>) e valida que mensagens de IPC vêm exclusivamente dessa janela específica, não de qualquer outra parte do app.
Validação de entrada	Campos de credenciais são validados antes de persistir no disco, tanto na criação quanto na edição de itens.

8. Módulos Nativos (Swift, iOS)

Dois módulos nativos escritos em Swift, empacotados como Expo Config Modules, dão ao app iOS acesso a funcionalidades que não existem em bibliotecas JavaScript genéricas:

Módulo	Função
<code>secure-vault-sync</code>	Gerencia autenticação e obtenção de tokens para o Google Drive de forma nativa no iOS.
<code>secure-vault-cloudkit</code>	Fala diretamente com o framework <code>CloudKit</code> nativo da Apple — leitura, escrita e exclusão de registros nos record types <code>VaultMeta</code> e <code>Credential</code> do container <code>iCloud.com.cortexistech.secpass</code> .

9. Auditoria de Segurança Realizada Nesta Fase do Projeto

O projeto passou por uma revisão de segurança adversarial completa, na qual foram identificadas e corrigidas 11 vulnerabilidades/lacunas, resumidas abaixo:

#	Achado	Correção
1	Reaproveitamento/rollback de envelopes antigos via backend comprometido	AAD vinculando <code>updatedAt</code> / <code>tombstone</code> ao AES-GCM (envelope v3, seção 4.2)
2	Um item corrompido derrubava a decifragem do cofre inteiro	Decifragem isolada por item, com try/catch individual
3	Bloqueio de login podia ser burlado matando o app no meio de uma tentativa	Persistência do estado de bloqueio antes de atualizar a UI
4	Tela de login ficava sem proteção contra captura de tela	Proteção movida para ativação incondicional, desde o boot da tela
5	Log de auditoria podia ser adulterado sem detecção	Selo HMAC encadeado (seção 7.1)
6	Cofre remoto de dispositivo novo era aceito sem nenhuma verificação humana	Resumo + confirmação explícita antes de decifrar (seção 6.4)
7	Desktop podia gravar segredos em texto claro se a criptografia do SO estivesse indisponível	<code>secureStore</code> agora recusa a operação em vez de degradar
8	Desktop não validava campos de credencial antes de persistir	Validação adicionada antes de toda escrita
9	Sandbox do Electron estava desabilitado em algumas janelas	Sandbox reativado (configuração padrão, mais segura)
10	Área de transferência do desktop não limpava a senha copiada	Auto-limpeza após 30s (igual ao mobile)
11	Regex de redação do Sentry tinha lacunas; exceção de rede local desnecessária no iOS	Regex ampliado; <code>NSAllowsLocalNetworking</code> desativado

10. Testes Automatizados

Suíte	Cobertura	Testes
Mobile (<code>__tests__</code> / , preset <code>jest-expo</code>)	Criptografia do cofre, storage/sync, merge, autenticação, throttle de login, geração de senha, componentes de UI, monitoramento de erros	210 testes / 17 arquivos
Desktop (<code>desktop/__tests__</code> / , Jest + Babel próprios)	Criptografia (porte do mobile, incluindo teste de compatibilidade byte-a-byte entre os dois), preferência de sync, storage/migração de backend, comparação em tempo constante	50 testes / 4 arquivos

O desktop precisou de uma configuração de teste própria porque seu código-fonte usa módulos ES (`import` / `export`) diretamente sob Node.js, o que exige uma transformação Babel (`@babel/preset-env`) que o preset padrão do mobile (pensado para React Native) não fornece.

11. Ferramentas de Engenharia e Processo de Build

11.1 Plugins de configuração customizados do Expo

Como o repositório do projeto vive dentro de uma pasta sincronizada pelo Google Drive (cujo caminho contém um espaço, "0 meu disco"), diversas ferramentas do ecossistema Expo/React Native/Sentry geram, internamente, comandos de shell sem as aspas necessárias para lidar com espaços em caminhos de arquivo — o que quebra builds nativos de forma silenciosa e intermitente. Dois mecanismos tornam essas correções permanentes e resistentes a regenerações do projeto:

- **patch-package** — corrige bugs de aspas dentro de pacotes do `node_modules` (`expo-constants` , `react-native` , `@sentry/react-native` , `expo-modules-jsi`), reaplicados automaticamente após todo `npm install` via um script `postinstall` .
- **Config plugin `withSafeShellScriptPaths.js`** — corrige o mesmo tipo de problema, mas no projeto Xcode (`.pbxproj`) que é gerado a cada `expo prebuild` — algo que um patch de `node_modules` não alcança, pois esse arquivo não existe até o momento do prebuild. Esse plugin foi validado contra uma regeneração completa e real do diretório `ios/` .
- **Config plugin `withCloudKitCapability.js`** — injeta as entitlements necessárias de iCloud/CloudKit no app iOS gerado.

11.2 Perfis de build (EAS)

O arquivo `eas.json` define perfis de build gerenciados pelo Expo Application Services para Android e iOS: `development` (para simulador/emulador, com ferramentas de debug), `preview` (build interno para testes) e `production-device` /produção (build final para lojas de aplicativo).

11.3 Empacotamento do desktop (electron-builder)

O app desktop é empacotado com `electron-builder` . Existem dois fluxos distintos, propositalmente separados:

Comando	Uso	Assinatura
<code>npm run dist</code>	Build local rápido para testar no próprio Mac do desenvolvedor	Nenhuma (<code>identity=null</code> , intencional — ver nota abaixo)
<code>npm run dist:release</code>	Build de distribuição real (<code>.dmg</code> / <code>.zip</code>)	Certificado Developer ID + notificação via variáveis de ambiente (<code>CSC_LINK</code> , <code>APPLE_ID</code> , etc.)

Detalhe técnico não óbvio: o build local (`dist`) usa a flag `-c.mac.identity=null` de propósito, para pular completamente a assinatura de código. Sem isso, o `electron-builder` tenta assinar (mesmo que apenas ad-hoc) e depois roda `codesign --verify --deep --strict` no aplicativo —

e esse comando falha porque o projeto vive numa pasta sincronizada pelo Google Drive, cujo provedor de arquivos marca itens com atributos estendidos que o `codesign` rejeita ("resource fork, Finder information, or similar detritus not allowed"). O fluxo de distribuição real (`dist:release`) mantém a assinatura completa (`hardenedRuntime` + entitlements), mas ainda não foi testado de ponta a ponta nesta fase por falta de um certificado Apple real disponível no ambiente de desenvolvimento.

12. Estado de Prontidão para Produção e Comercialização

12.1 O que já está pronto

- Criptografia E2E completa e revisada adversarialmente (11 correções de segurança aplicadas)
- Sincronização funcional via Google Drive e iCloud, com escolha manual e migração
- Log de auditoria à prova de adulteração
- Suítes de teste automatizado no mobile (210) e no desktop (50), todas passando
- Mecanismos de build reproduzíveis (patches + plugins) para o ambiente específico deste projeto
- Estrutura de assinatura/notarização do desktop pronta para receber um certificado real

12.2 O que falta (bloqueadores técnicos)

- **Extensão de AutoFill no iOS** (preencher senhas em outros apps/Safari) — deliberadamente adiada para um projeto separado, ainda não iniciada.
- **Assinatura/notarização real do desktop** — a estrutura existe (seção 11.3), mas nunca rodou de ponta a ponta com um certificado Apple Developer ID de verdade.
- **Dependências do desktop com vulnerabilidades conhecidas** — `electron` / `electron-builder` / `extract-zip` / `node-tar` têm alertas de segurança (alto/crítico) no `npm audit`, que exigiriam uma atualização com potencial quebra de compatibilidade — decisão ainda não tomada com o usuário.
- **Teste real em dois dispositivos físicos** — os testes automatizados cobrem a lógica isoladamente (com mocks), mas o fluxo completo de duas contas Google Drive/iCloud reais sincronizando entre si nunca foi validado manualmente neste ambiente.

12.3 O que falta (fora do código — jurídico/negócio)

- Revisão jurídica da política de privacidade e termos de serviço para o mercado-alvo
- Definição de modelo de negócio (gratuito, assinatura, etc.) e infraestrutura de cobrança, se aplicável
- Processo de submissão às lojas (App Store/Play Store) — contas de desenvolvedor, revisão de conteúdo, capturas de tela, descrição
- Suporte ao usuário e canal de reporte de vulnerabilidades de segurança

13. Limitações Conhecidas e Decisões de Design

13.1 Confiança no primeiro sync ("trust on first sync")

Como descrito na seção 6.4, um dispositivo completamente novo não tem como verificar criptograficamente, por si só, que o cofre remoto que está baixando é genuíno e atual — essa é uma limitação estrutural de qualquer sincronização E2E sem um servidor que arbitra ordem/autenticidade (o mesmo desafio que motiva a infraestrutura de servidor de apps como o Signal). A mitigação implementada (mostrar um resumo e pedir confirmação humana) reduz o risco prático, mas não o elimina por completo — resolver isso de forma definitiva exigiria adicionar infraestrutura de servidor própria, o que está fora do escopo atual do projeto.

13.2 Ambiente de desenvolvimento específico deste projeto

Diversas correções de build (patches, plugins, flags de assinatura) existem especificamente porque este projeto é desenvolvido dentro de uma pasta sincronizada pelo Google Drive. Se o projeto for movido para fora dessa pasta no futuro, essas correções deixam de ser necessárias (mas continuam inofensivas caso permaneçam).

13.3 Ferramentas ausentes no ambiente local

O binário `cmake` não está disponível no `PATH` deste ambiente, o que impede rodar um build iOS completo (`pod install`) até o fim localmente — não é um problema introduzido pelo código do projeto, apenas uma dependência de ferramenta de sistema que precisa ser instalada separadamente.